Laboratory Work 2

Course

Microcontroller Systems

Title

Clock Configuration, SysTick 10 kHz and Output Frequency Measurement

Objective

To extend Lab 1 firmware with system clock setup logic, periodic interrupt generation, and measurable output signal toggling.

Variant

Variant: **5** Required frequency by task table: **80 MHz**.

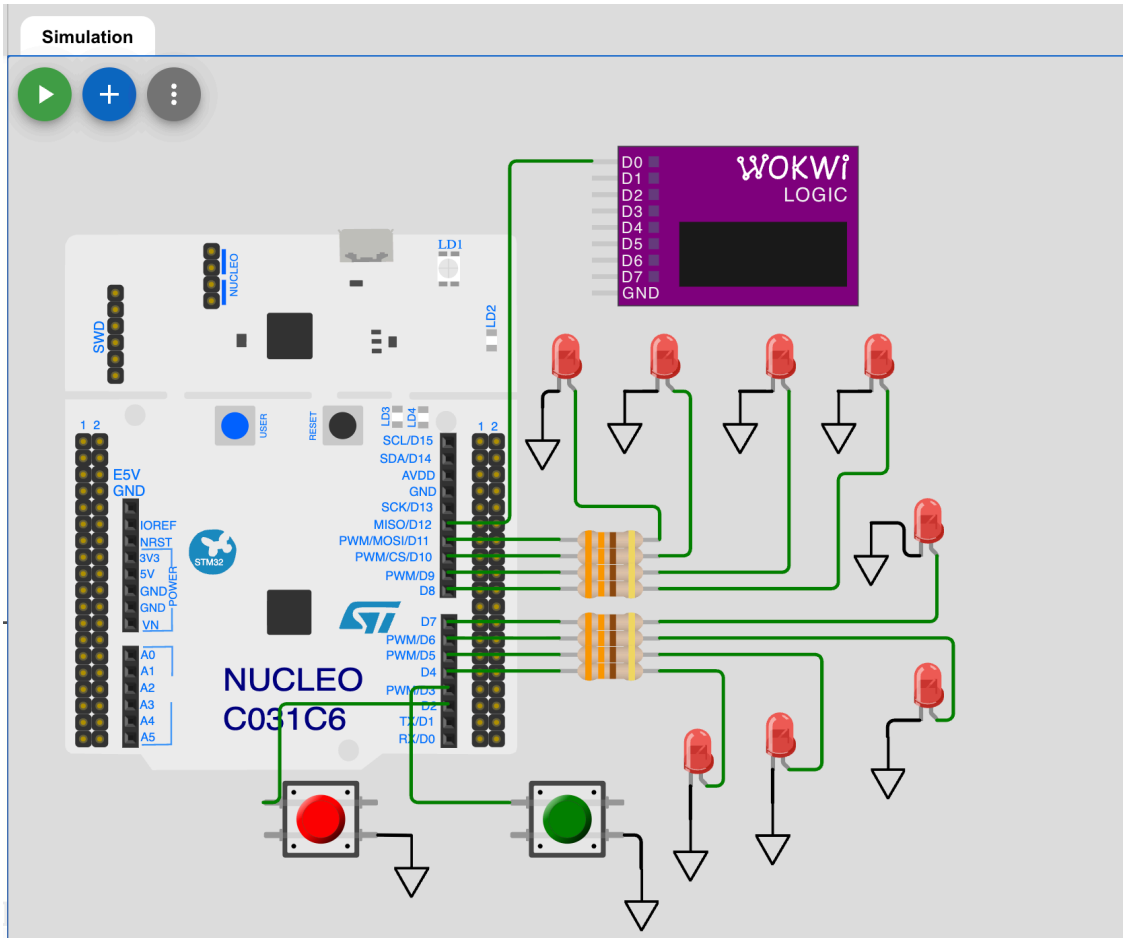
Task Requirements

1. Configure flash latency.
2. Configure PLL source and PLL factors.
3. Enable PLL and switch main clock source.
4. Start SysTick timer at 10 kHz.
5. Toggle GPIO pin inside SysTick handler.
6. Measure output signal frequency in simulation tools.

Schematic / Simulation Setup

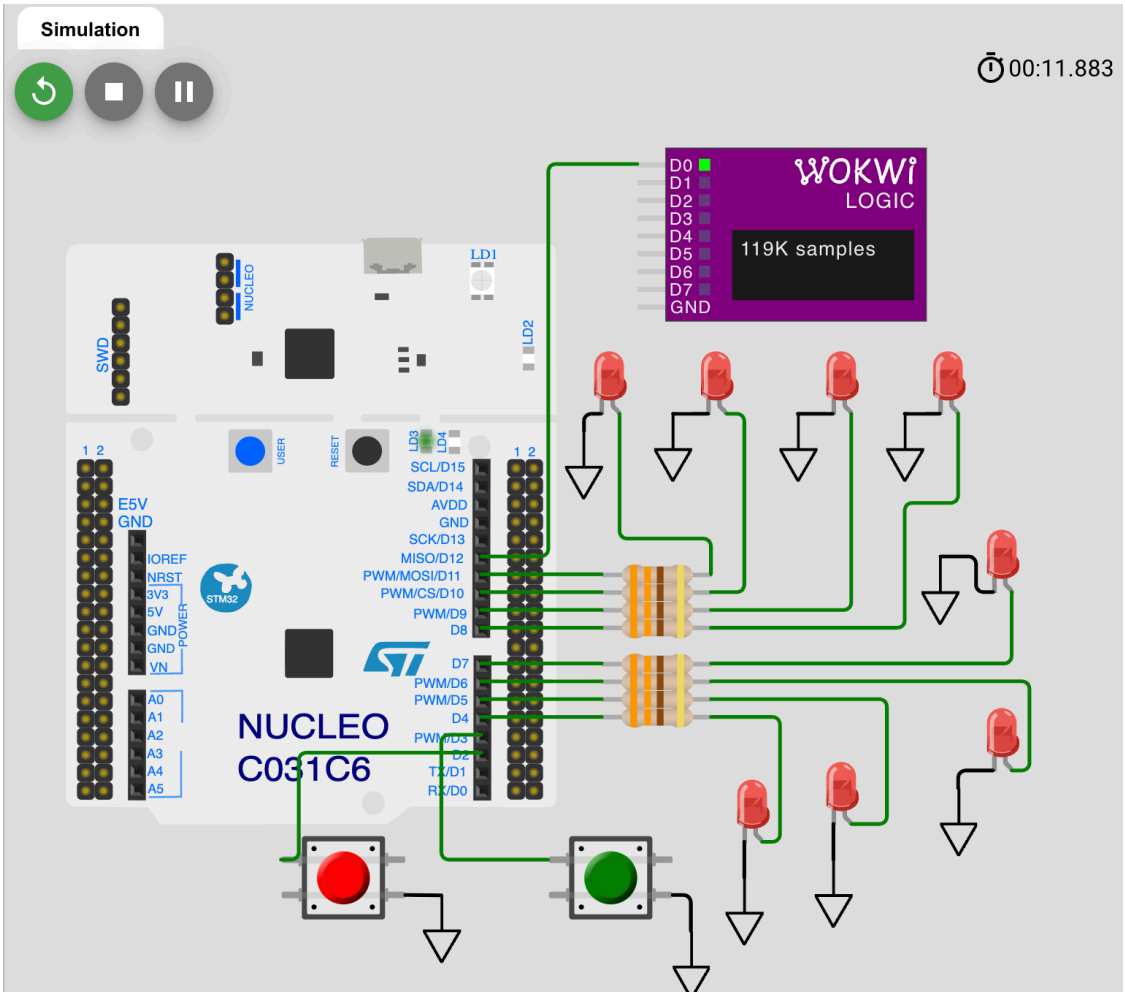
Runtime demonstration was assembled and executed in Wokwi.

- Scheme view:



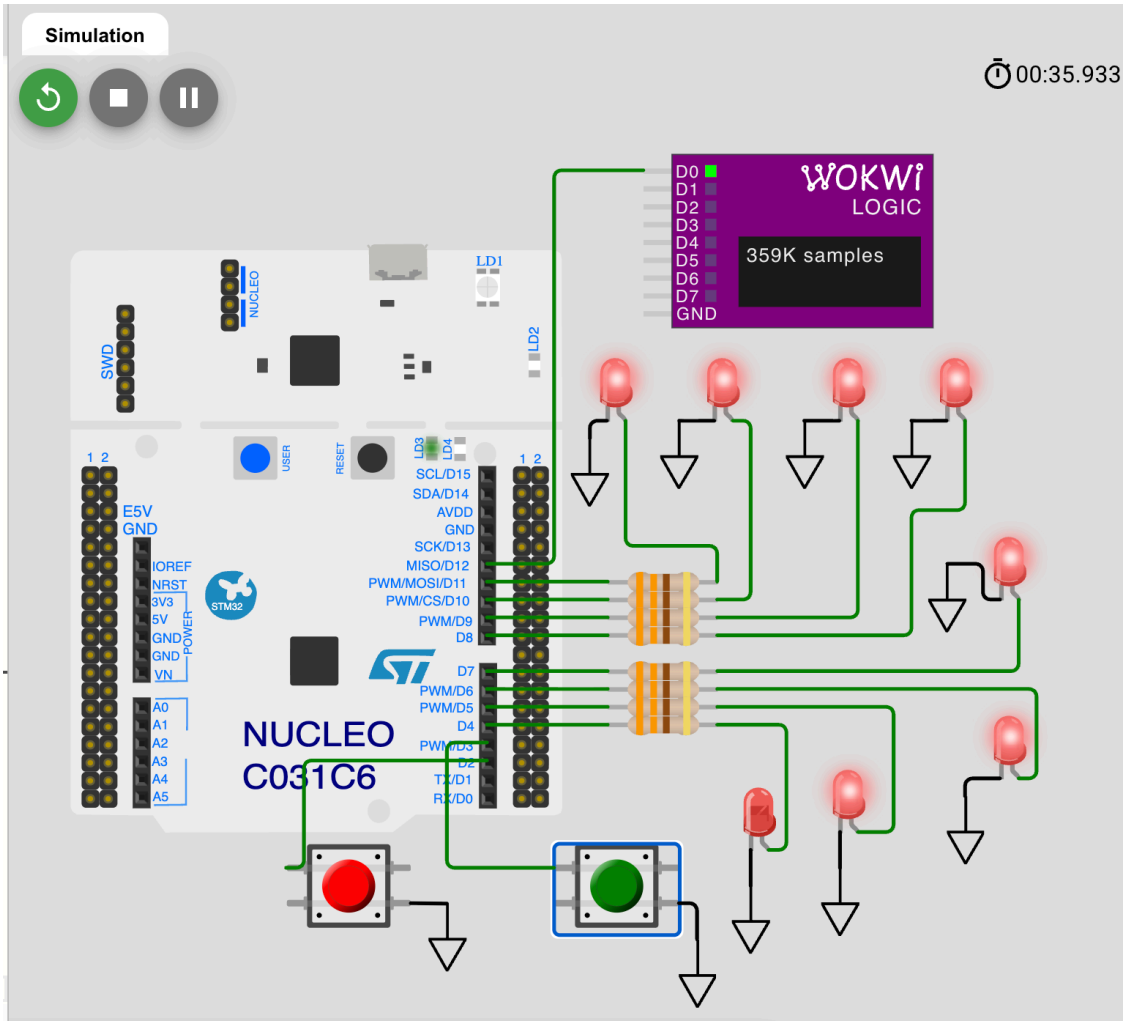
Lab 2 Wokwi scheme

- Logic analyzer running view:



Lab 2 logic analyzer

- Frequency demonstration with active runtime state:



Lab 2 frequency value

Source Code

Primary STM32 implementation file: - `Lab2_main.c`

Wokwi runtime analogue sketch file: - `Wokwi_Lab2_sketch.ino`

Calculations (Variant 5)

Required target frequency from variant table: - `f_SYSCLK = 80 MHz`

PLL setup used in STM32 code: - `HSI = 16 MHz` - `PLLM = 16` - `PLLN = 320` - `PLLP = 4`

Check: 1. `f_VCO_IN = 16 / 16 = 1 MHz` 2. `f_VCO_OUT = 1 * 320 = 320 MHz` 3. `f_SYSCLK = 320 / 4 = 80 MHz`

SysTick setup: - required interrupt frequency: `10 kHz` - reload value: `LOAD = 80,000,000 / 10,000 - 1 = 7999`

Output signal behavior: - output pin toggles on each tick, - expected square-wave fundamental frequency is approximately: - `f_OUT ≈ 10 kHz / 2 = 5 kHz`

Verification Notes

1. LED/button behavior from Lab 1 remains operational.
2. Output pin signal is generated continuously.
3. Logic analyzer confirms periodic toggling.

Code Listing (`main.c`)

```c
#include "stm32f4xx.h"
#include <stdint.h>

#ifndef LAB_VARIANT
#define LAB_VARIANT 5u
#endif

#define BTN_UP_PIN      (0u)    // PA0
#define BTN_DOWN_PIN    (1u)    // PA1
#define LED_MASK_8BIT   (0xFFu) // PC0..PC7
#define FREQ_OUT_PIN    (0u)    // PB0 for oscilloscope

static volatile uint8_t g_counter = 0u;

static uint32_t target_freq_mhz_from_variant(uint32_t variant)
{
    if (variant < 1u) {
        return 84u;
    }
    if (variant > 41u) {
        return 44u;
    }
    return 85u - variant;
}
```

```c
static void delay_cycles(volatile uint32_t n)
{
    while (n--) {
        __NOP();
    }
}

static uint8_t button_pressed(uint32_t pin)
{
    return ((GPIOA->IDR & (1u << pin)) == 0u) ? 1u : 0u;
}

static void leds_write(uint8_t value)
{
    uint32_t odr = GPIOC->ODR;
    odr &= ~LED_MASK_8BIT;
    odr |= value;
    GPIOC->ODR = odr;
}

static void gpio_init(void)
{
    RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN | RCC_AHB1ENR_GPIOBEN |
RCC_AHB1ENR_GPIOCEN;
    (void)RCC->AHB1ENR;

    // PC0..PC7 as LEDs.
    GPIOC->MODER &= ~(0xFFFFu);
    GPIOC->MODER |=  (0x5555u);
    GPIOC->OTYPER &= ~(LED_MASK_8BIT);
    GPIOC->PUPDR  &= ~(0xFFFFu);

    // PA0/PA1 as button inputs with pull-up.
    GPIOA->MODER &= ~((3u << (BTN_UP_PIN * 2u)) | (3u << (BTN_DOWN_PIN
* 2u)));
    GPIOA->PUPDR &= ~((3u << (BTN_UP_PIN * 2u)) | (3u << (BTN_DOWN_PIN
* 2u)));
    GPIOA->PUPDR |=  ((1u << (BTN_UP_PIN * 2u)) | (1u << (BTN_DOWN_PIN
* 2u)));
```

```
    // PB0 as output for frequency observation.
    GPIOB->MODER &= ~(3u << (FREQ_OUT_PIN * 2u));
    GPIOB->MODER |=  (1u << (FREQ_OUT_PIN * 2u));
    GPIOB->OTYPER &= ~(1u << FREQ_OUT_PIN);
    GPIOB->PUPDR  &= ~(3u << (FREQ_OUT_PIN * 2u));
}

static void set_flash_latency(uint32_t sysclk_mhz)
{
    uint32_t latency;

    if (sysclk_mhz <= 30u) {
        latency = FLASH_ACR_LATENCY_0WS;
    } else if (sysclk_mhz <= 60u) {
        latency = FLASH_ACR_LATENCY_1WS;
    } else {
        latency = FLASH_ACR_LATENCY_2WS;
    }

    FLASH->ACR = FLASH_ACR_ICEN | FLASH_ACR_DCEN | FLASH_ACR_PRFTEN |
latency;
    while ((FLASH->ACR & FLASH_ACR_LATENCY) != latency) {
    }
}

static void clock_init_pll_hsi(uint32_t target_mhz)
{
    uint32_t pllm = 16u;
    uint32_t pllp = 4u;
    uint32_t plln;

    // Keep HSI on.
    RCC->CR |= RCC_CR_HSION;
    while ((RCC->CR & RCC_CR_HSIRDY) == 0u) {
    }

    // Temporarily switch SYSCLK to HSI before touching PLL.
    RCC->CFGR &= ~RCC_CFGR_SW;
    while ((RCC->CFGR & RCC_CFGR_SWS) != RCC_CFGR_SWS_HSI) {
    }
```

```
    RCC->CR &= ~RCC_CR_PLLON;
    while ((RCC->CR & RCC_CR_PLLRDY) != 0u) {
    }

    // Choose PLLP so PLLN stays in valid range.
    if ((target_mhz * 4u) >= 192u) {
        pllp = 4u;
    } else {
        pllp = 6u;
    }
    plln = target_mhz * pllp;

    set_flash_latency(target_mhz);

    // AHB = SYSCLK; APB1 <= 42 MHz; APB2 <= 84 MHz.
    RCC->CFGR &= ~(RCC_CFGR_HPRE | RCC_CFGR_PPRE1 | RCC_CFGR_PPRE2);
    RCC->CFGR |= RCC_CFGR_HPRE_DIV1 | RCC_CFGR_PPRE1_DIV2 |
RCC_CFGR_PPRE2_DIV1;

    RCC->PLLCFGR = (pllm & RCC_PLLCFGR_PLLM)
                 | ((plln << RCC_PLLCFGR_PLLN_Pos) & RCC_PLLCFGR_PLLN)
                 | ((((pllp / 2u) - 1u) << RCC_PLLCFGR_PLLP_Pos) &
RCC_PLLCFGR_PLLP)
                 | RCC_PLLCFGR_PLLSRC_HSI;

    RCC->CR |= RCC_CR_PLLON;
    while ((RCC->CR & RCC_CR_PLLRDY) == 0u) {
    }

    RCC->CFGR &= ~RCC_CFGR_SW;
    RCC->CFGR |= RCC_CFGR_SW_PLL;
    while ((RCC->CFGR & RCC_CFGR_SWS) != RCC_CFGR_SWS_PLL) {
    }
}

static void systick_init_10khz(uint32_t core_clk_hz)
{
    SysTick->LOAD = (core_clk_hz / 10000u) - 1u;
    SysTick->VAL = 0u;
```

```
    SysTick->CTRL = SysTick_CTRL_CLKSOURCE_Msk
                  | SysTick_CTRL_TICKINT_Msk
                  | SysTick_CTRL_ENABLE_Msk;
}

void SysTick_Handler(void)
{
    // Toggle PB0 at each SysTick interrupt.
    GPIOB->ODR ^= (1u << FREQ_OUT_PIN);
}

int main(void)
{
    const uint32_t target_mhz =
target_freq_mhz_from_variant(LAB_VARIANT);
    uint8_t up_prev = 0u;
    uint8_t down_prev = 0u;

    clock_init_pll_hsi(target_mhz);
    gpio_init();
    leds_write(g_counter);

    // Required by task: SysTick at 10 kHz.
    systick_init_10khz(target_mhz * 1000000u);

    for (;;) {
        uint8_t up_now = button_pressed(BTN_UP_PIN);
        uint8_t down_now = button_pressed(BTN_DOWN_PIN);

        if ((up_now != 0u) && (up_prev == 0u)) {
            g_counter++;
            leds_write(g_counter);
            delay_cycles(120000u);
        }

        if ((down_now != 0u) && (down_prev == 0u)) {
            g_counter--;
            leds_write(g_counter);
            delay_cycles(120000u);
        }
```

```
        up_prev = up_now;
        down_prev = down_now;
    }
}
```

Conclusion

The second laboratory task objectives were achieved: clock-configuration logic was implemented, 10 kHz periodic timing was established, and output frequency behavior was demonstrated and captured.